

Code for FOLLOWER

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from sensor_msgs.msg import LaserScan
from geometry_msgs.msg import Twist
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
import numpy as np
import math
import time

class Follower(Node):
    def __init__(self):
        super().__init__('follower')

        # — PARAMS (following logic UNCHANGED) —————
        self.MAX_TURN      = 0.35
        self.MAX_SPEED     = 0.20
        self.ANGLE_DEAD    = 12.0
        self.LOST_TIMEOUT  = 2.0
        self.BACK_SPEED    = 0.08

        # — DISTANCE PARAMS
        self.FOLLOW_DIST   = 0.7 # target distance (m)
        self.TOO_CLOSE     = 0.5 # reverse if closer than this
        self.TOO_FAR       = 0.9 # move forward if further than this

        # — SEARCH PARAMS
        self.SEARCH_START  = 3.0
        self.SEARCH_STOP   = 13.0

        # — STATE
        self._angle        = None
        self._front_dist    = None
        self._last_seen    = time.time()
```

— ROS2

```
sensor_qos = QoSProfile(
    reliability=ReliabilityPolicy.BEST_EFFORT,
    history=HistoryPolicy.KEEP_LAST,
    depth=10
)

self.angle_sub = self.create_subscription(
    Float32, '/person_angle', self.angle_cb, 10)

self.scan_sub = self.create_subscription(
    LaserScan, '/scan', self.scan_cb, sensor_qos)

self.cmd_pub = self.create_publisher(Twist, '/cmd_vel', 10)
self.timer = self.create_timer(0.1, self.loop)

self.get_logger().info('=== Follower Ready ===')
self.get_logger().info(f'Follow distance: {self.FOLLOW_DIST}m')
```

— ANGLE CALLBACK (UNCHANGED)

```
def angle_cb(self, msg):
    if not math.isnan(msg.data):
        self._angle = msg.data
        self._last_seen = time.time()
    else:
        if time.time() - self._last_seen > self.LOST_TIMEOUT:
            self._angle = None
```

— LIDAR CALLBACK

```
def scan_cb(self, msg):
    front = []
    for i, d in enumerate(msg.ranges):
        if not np.isfinite(d) or d < 0.1 or d > 10.0:
            continue
        angle_rad = msg.angle_min + i * msg.angle_increment
        angle_deg = np.degrees(angle_rad) % 360

        if angle_deg <= 15 or angle_deg >= 345:
            front.append(d)
    if front:
```

```

        self._front_dist = float(np.percentile(front, 10))
    if front:
        self._front_dist = float(np.percentile(front, 10))
        self.get_logger().info(
            f'LIDAR front readings: {len(front)} rays,
dist:{self._front_dist:.2f}m',
            throttle_duration_sec=1.0
        )

```

— MAIN LOOP

```

def loop(self):
    cmd = Twist()
    elapsed = time.time() - self._last_seen

```

— SEARCH MODE

```

if self._angle is None:
    if elapsed < self.SEARCH_START:
        self.get_logger().info(
            'Person lost – waiting...',
            throttle_duration_sec=2.0)
    elif elapsed < self.SEARCH_STOP:
        cmd.angular.z = 0.25
        self.get_logger().info(
            f'Searching... ({elapsed:.0f}s)',
            throttle_duration_sec=2.0)
    else:
        self.get_logger().info(
            'Waiting for person...',
            throttle_duration_sec=3.0)
self.cmd_pub.publish(cmd)
return

```

```

age  = time.time() - self._last_seen
angle = self._angle

```

— TURNING (COMPLETELY UNCHANGED)

```

if abs(angle) > self.ANGLE_DEAD:
    cmd.angular.z = float(np.clip(
        np.radians(angle) * 0.8,
        -self.MAX_TURN, self.MAX_TURN
    ))

```

```
# — DISTANCE CONTROL (LIDAR)
```

```
if age < 1.0:
    turn_factor = 1.0 - min(abs(angle) / 60.0, 1.0)

    if self._front_dist is not None and abs(angle) < 20.0:
        # Facing person – LIDAR controls distance
        dist = self._front_dist
        if dist < self.TOO_CLOSE:
            # Too close – reverse
            cmd.linear.x = -self.BACK_SPEED
            self.get_logger().info(
                f'Too close {dist:.2f}m – reversing',
                throttle_duration_sec=1.0)
        elif dist > self.TOO_FAR:
            # Too far – move forward
            cmd.linear.x = self.MAX_SPEED * turn_factor
        else:
            # Good distance – hold position
            cmd.linear.x = 0.0

    else:
        # Not facing person yet – gentle forward
        cmd.linear.x = self.MAX_SPEED * 0.5 * turn_factor

else:
    cmd.linear.x = 0.0

self.get_logger().info(
    f'angle:{angle:.1f} '
    f'dist:{self._front_dist:.2f}m '
    f'spd:{cmd.linear.x:.2f} '
    f'turn:{cmd.angular.z:.2f}'
    if self._front_dist else
    f'angle:{angle:.1f} dist:-- '
    f'spd:{cmd.linear.x:.2f} '
    f'turn:{cmd.angular.z:.2f}'
)
self.cmd_pub.publish(cmd)

def destroy_node(self):
    try:
```

```

        self.cmd_pub.publish(Twist())
    except Exception:
        pass
    super().destroy_node()

def main():
    rclpy.init()
    node = Follower()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Code for camera:

```

import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32
from ultralytics import YOLO
from flask import Flask, Response, jsonify
from flask_cors import CORS
import cv2
import threading
import subprocess
import time

app = Flask(__name__)
CORS(app) # allows website to talk to Pi from any origin

latest_frame = None
frame_lock = threading.Lock()
emergency_stop = False # shared flag

```

```

class CameraNode(Node):
    def __init__(self):
        super().__init__('camera_node')
        self.pub = self.create_publisher(Float32, '/person_angle', 10)
        self.emergency_pub = self.create_publisher(
            __import__('geometry_msgs.msg', fromlist=['Twist']).Twist,
            '/cmd_vel', 10
        )

        self.get_logger().info('Loading YOLO...')
        self.model = YOLO('yolov8n.pt')
        self.get_logger().info('YOLO ready!')

        self.HFOV      = 78.0
        self.frame_cx   = 320.0
        self.MIN_CONF   = 0.5
        self.cap         = None

        self._open_camera()
        self.timer = self.create_timer(0.1, self.detect)

    def _open_camera(self):
        subprocess.run(['sudo', 'fuser', '-k', '/dev/video0'],
                       capture_output=True)
        time.sleep(1.0)

        for attempt in range(10):
            cap = cv2.VideoCapture(0)
            cap.set(cv2.CAP_PROP_FOURCC,
                   cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
            cap.set(cv2.CAP_PROP_FPS, 10)

            if cap.isOpened():
                ret, frame = cap.read()
                if ret and frame is not None:
                    self.cap = cap
                    self.get_logger().info('Camera ready!')
                    return
            cap.release()
            self.get_logger().warn(f'Camera attempt {attempt+1} failed...')
            time.sleep(2.0)

```

```

        self.get_logger().error('Camera failed!')

def detect(self):
    global latest_frame, emergency_stop

    # If emergency stop – publish zero velocity repeatedly
    if emergency_stop:
        from geometry_msgs.msg import Twist
        self.emergency_pub.publish(Twist())
        return

    if self.cap is None or not self.cap.isOpened():
        self._open_camera()
        return

    ret, frame = self.cap.read()
    if not ret:
        self.cap.release()
        self.cap = None
        return

    results = self.model(
        frame, imgsz=128, classes=[0],
        conf=self.MIN_CONF, verbose=False
    )

    best_area = 0
    best_angle = None

    for box in results[0].boxes:
        conf = float(box.conf[0])
        x1, y1, x2, y2 = map(float, box.xyxy[0])
        area = (x2-x1) * (y2-y1)
        if area > best_area:
            best_area = area
            cx = (x1+x2) / 2.0
            best_angle = -1 * ((cx - self.frame_cx) / self.frame_cx) *
(self.HFOV / 2.0)

            cv2.rectangle(frame,
                (int(x1),int(y1)), (int(x2),int(y2)),
                (0,255,0), 2)

```

[illegible]

```

        yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n' +
               jpeg.tobytes() + b'\r\n')

@app.route('/')
def index():
    return '''<html><body style="background:black;margin:0">

</body></html>'''

@app.route('/video')
def video():
    return Response(generate(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/emergency_stop', methods=['POST'])
def stop():
    global emergency_stop
    emergency_stop = True
    return jsonify({'status': 'stopped'})

@app.route('/resume', methods=['POST'])
def resume():
    global emergency_stop
    emergency_stop = False
    return jsonify({'status': 'resumed'})

@app.route('/health')
def health():
    return jsonify({'status': 'online', 'emergency': emergency_stop})

def main():
    # Install flask-cors if needed
    subprocess.run(['pip3', 'install', 'flask-cors', '--user', '-q'])

    flask_thread = threading.Thread(
        target=lambda: app.run(host='0.0.0.0', port=5001),
        daemon=True
    )
    flask_thread.start()

    rclpy.init()
    node = CameraNode()

```

```

import socket
ip = socket.gethostbyname(socket.gethostname())
print(f'\n Stream:          http://{ip}:5000/video')
print(f' Emergency stop: POST http://{ip}:5000/emergency_stop')
print(f' Resume:           POST http://{ip}:5000/resume\n')

try:
    rclpy.spin(node)
except KeyboardInterrupt:
    pass
finally:
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Code for CONTROLLER:

```

import threading, time, socket, subprocess
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan, BatteryState
from rclpy.qos import QoSProfile, ReliabilityPolicy, HistoryPolicy
from flask import Flask, Response, jsonify, request
from flask_cors import CORS
import cv2, numpy as np

app = Flask(__name__)
CORS(app)

# — shared state


---


latest_frame      = None
frame_lock        = threading.Lock()

```

```

person_angle      = None
angle_lock        = threading.Lock()
emergency_stopped = False
cam_running       = False
follower_running  = False
battery_level     = 0.0

# — CAMERA THREAD (no YOLO – laptop handles detection) —
def camera_loop():
    global latest_frame, cam_running

    subprocess.run(['sudo', 'fuser', '-k', '/dev/video0'],
                    capture_output=True)
    time.sleep(1.0)

    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc('M', 'J', 'P', 'G'))
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    cap.set(cv2.CAP_PROP_FPS, 10)

    if not cap.isOpened():
        print('Camera: failed!')
        cam_running = False
        return

    print('Camera: open! YOLO running on laptop.')

    while cam_running:
        ret, frame = cap.read()
        if not ret:
            continue
        with frame_lock:
            latest_frame = frame.copy()

    cap.release()
    print('Camera: stopped')

# — FOLLOWER THREAD (exact same as follower_standalone.py) —
def follower_loop():
    global follower_running, emergency_stopped

    rclpy.init()

```

```

class Follower(Node):
    def __init__(self):
        super().__init__('follower')

        self.MAX_TURN      = 0.35
        self.MAX_SPEED      = 0.20
        self.ANGLE_DEAD     = 12.0
        self.LOST_TIMEOUT   = 2.0
        self.BACK_SPEED     = 0.08
        self.FOLLOW_DIST    = 0.7
        self.TOO_CLOSE      = 0.5
        self.TOO_FAR        = 0.9
        self.SEARCH_START   = 3.0
        self.SEARCH_STOP    = 13.0

        self._front_dist = None
        self._last_seen  = time.time()

        sensor_qos = QoSProfile(
            reliability=ReliabilityPolicy.BEST_EFFORT,
            history=HistoryPolicy.KEEP_LAST,
            depth=10
        )

        self.scan_sub    = self.create_subscription(
            LaserScan, '/scan', self.scan_cb, sensor_qos)
        self.battery_sub = self.create_subscription(
            BatteryState, '/battery_state', self.battery_cb, 10)
        self.cmd_pub     = self.create_publisher(Twist, '/cmd_vel', 10)
        self.timer       = self.create_timer(0.1, self.loop)

        self.get_logger().info('=== Follower Ready ===')

    def scan_cb(self, msg):
        front = []
        for i, d in enumerate(msg.ranges):
            if not np.isfinite(d) or d < 0.1 or d > 10.0:
                continue
            deg = (np.degrees(
                msg.angle_min + i * msg.angle_increment
            )) % 360
            if deg <= 15 or deg >= 345:

```

```

        front.append(d)
    if front:
        self._front_dist = float(np.percentile(front, 10))
        self.get_logger().info(
            f'LIDAR front: {len(front)} rays,
dist:{self._front_dist:.2f}m',
            throttle_duration_sec=1.0)

def battery_cb(self, msg):
    global battery_level
    pct = msg.percentage
    if pct > 1.0:
        battery_level = round(min(pct, 100.0), 1)
    else:
        battery_level = round(pct * 100, 1)

def loop(self):
    cmd = Twist()
    elapsed = time.time() - self._last_seen

    if emergency_stopped:
        self.cmd_pub.publish(cmd)
        return

    with angle_lock:
        angle = person_angle

    if angle is not None:
        self._last_seen = time.time()
        elapsed = 0.0

    if angle is None:
        if self.SEARCH_START < elapsed < self.SEARCH_STOP:
            cmd.angular.z = 0.25
            self.get_logger().info(
                f'Searching... ({elapsed:.0f}s)',
                throttle_duration_sec=2.0)
        else:
            self.get_logger().info(
                'Waiting for person...',
                throttle_duration_sec=3.0)
        self.cmd_pub.publish(cmd)
    return

```

```

age = time.time() - self._last_seen

if abs(angle) > self.ANGLE_DEAD:
    cmd.angular.z = float(np.clip(
        np.radians(angle) * 0.8,
        -self.MAX_TURN, self.MAX_TURN
    ))

if age < 1.0:
    tf = 1.0 - min(abs(angle) / 60.0, 1.0)
    if self._front_dist is not None and abs(angle) < 20.0:
        dist = self._front_dist
        if dist < self.TOO_CLOSE:
            cmd.linear.x = -self.BACK_SPEED
            self.get_logger().info(
                f'Too close {dist:.2f}m - reversing',
                throttle_duration_sec=1.0)
        elif dist > self.TOO_FAR:
            cmd.linear.x = self.MAX_SPEED * tf
        else:
            cmd.linear.x = 0.0
    else:
        cmd.linear.x = self.MAX_SPEED * 0.5 * tf
else:
    cmd.linear.x = 0.0

self.get_logger().info(
    f'angle:{angle:.1f} '
    f'dist:{self._front_dist:.2f}m '
    f'spd:{cmd.linear.x:.2f} '
    f'turn:{cmd.angular.z:.2f}'
    if self._front_dist else
    f'angle:{angle:.1f} dist:-- '
    f'spd:{cmd.linear.x:.2f} '
    f'turn:{cmd.angular.z:.2f}'
)
self.cmd_pub.publish(cmd)

def destroy_node(self):
    try:
        self.cmd_pub.publish(Twist())
    except Exception:

```

```

        pass
    super().destroy_node()

node = Follower()
while follower_running and rclpy.ok():
    rclpy.spin_once(node, timeout_sec=0.1)
node.destroy_node()
rclpy.shutdown()

# — VIDEO STREAM


---


def generate():
    while True:
        with frame_lock:
            if latest_frame is None:
                time.sleep(0.05)
                continue
            _, jpeg = cv2.imencode('.jpg', latest_frame,
                                   [cv2.IMWRITE_JPEG_QUALITY, 70])
            yield (b'--frame\r\nContent-Type: image/jpeg\r\n\r\n'
                  + jpeg.tobytes() + b'\r\n')

@app.route('/video')
def video():
    return Response(generate(),
                    mimetype='multipart/x-mixed-replace; boundary=frame')

# — ENDPOINTS


---


@app.route('/start_camera', methods=['POST'])
def start_camera():
    global cam_running
    if not cam_running:
        cam_running = True
        threading.Thread(target=camera_loop, daemon=True).start()
    return jsonify({'status': 'camera started'})

@app.route('/stop_camera', methods=['POST'])
def stop_camera():
    global cam_running
    cam_running = False
    return jsonify({'status': 'camera stopped'})

```

```

@app.route('/start_follower', methods=['POST'])
def start_follower():
    global follower_running
    if not follower_running:
        follower_running = True
        threading.Thread(target=follower_loop, daemon=True).start()
    return jsonify({'status': 'follower started'})

@app.route('/stop_follower', methods=['POST'])
def stop_follower():
    global follower_running
    follower_running = False
    return jsonify({'status': 'follower stopped'})

@app.route('/emergency_stop', methods=['POST'])
def do_emergency_stop():
    global emergency_stopped, cam_running, follower_running
    emergency_stopped = True
    cam_running = False
    follower_running = False
    return jsonify({'status': 'emergency stopped'})

@app.route('/resume', methods=['POST'])
def resume():
    global emergency_stopped
    emergency_stopped = False
    return jsonify({'status': 'resumed'})

@app.route('/set_angle', methods=['POST'])
def set_angle():
    global person_angle
    data = request.get_json()
    angle = data.get('angle', None)
    with angle_lock:
        person_angle = angle
    return jsonify({'status': 'ok'})

@app.route('/health')
def health():
    return jsonify({
        'status': 'online',
        'cam': cam_running,
        'following': follower_running,
    })

```

```

        'emergency': emergency_stopped,
        'battery': battery_level
    })

# — MAIN

if __name__ == '__main__':
    ip = socket.gethostname(socket.gethostname())
    print(f'\nAVAA Controller → http://{ip}:5000')
    print('Endpoints: /start_camera /stop_camera /start_follower')
    print('                /stop_follower /emergency_stop /resume /video /health
/set_angle\n')
    app.run(host='0.0.0.0', port=5000)

```

Code ON LAPTOP:

```

import cv2
import requests
import time
import threading
from ultralytics import YOLO
from deep_sort_realtime.deepsort_tracker import
DeepSort
from flask import Flask, jsonify
from flask_cors import CORS

PI_IP      = '192.168.0.135'
STREAM     = f'http://{PI_IP}:5000/video'
ANGLE_URL  = f'http://{PI_IP}:5000/set_angle'
LAPTOP_PORT = 5002

# — shared latest frame

latest_raw = None
raw_lock   = threading.Lock()

```

```

stream_alive = True

def frame_reader():
    """Constantly reads stream, always keeps only
latest frame."""
    global latest_raw, stream_alive
    cap = cv2.VideoCapture(STREAM)
    cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
    while stream_alive:
        ret, frame = cap.read()
        if not ret:
            print('Stream lost, retrying...')
            time.sleep(1.0)
            cap = cv2.VideoCapture(STREAM)
            cap.set(cv2.CAP_PROP_BUFFERSIZE, 1)
            continue
        with raw_lock:
            latest_raw = frame # always overwrite —
no queue buildup
        cap.release()

# Start reader thread immediately
threading.Thread(target=frame_reader,
daemon=True).start()

# — Flask for laptop endpoints


---


app_laptop = Flask(__name__)
CORS(app_laptop)

target_id = None
target_lock = threading.Lock()

```

```
@app_laptop.route('/reset_target', methods=['POST'])
def reset_target():
    global target_id
    with target_lock:
        target_id = None
    print('Target reset!')
    return jsonify({'status': 'reset'})

@app_laptop.route('/health')
def health():
    return jsonify({'status': 'online', 'target':
target_id})

threading.Thread(
    target=lambda: app_laptop.run(host='0.0.0.0',
port=LAPTOP_PORT),
    daemon=True
).start()

# — Models
```

```
print('Loading YOLO + DeepSORT...')
model = YOLO('yolov8n.pt')
tracker = DeepSort(
    max_age=60,
    n_init=1,
    embedder='mobilenet',
    embedder_gpu=False
)
print('Ready!')

HFOV = 78.0
frame_cx = 320.0
```

```
MIN_CONF = 0.5

# — Main processing loop

print(f'Processing stream from {STREAM}')
print('Waiting for first frame...')

while True:
    # Grab latest frame — non-blocking
    with raw_lock:
        if latest_raw is None:
            time.sleep(0.01)
            continue
        frame = latest_raw.copy()

    start = time.time()

    # YOLO
    results = model(frame, imgsz=160, classes=[0],
                    conf=MIN_CONF, verbose=False)

    detections = []
    for box in results[0].boxes:
        x1, y1, x2, y2 = map(float, box.xyxy[0])
        conf = float(box.conf[0])
        detections.append([x1, y1, x2-x1, y2-y1],
conf, 0))

    # DeepSORT
    tracks = tracker.update_tracks(detections,
frame=frame)

    best_angle = None
```

```

best_area = 0

with target_lock:
    current_target = target_id

for track in tracks:
    if not track.is_confirmed():
        continue
    tid = track.track_id
    ltrb = track.to_ltrb()
    x1, y1, x2, y2 = map(int, ltrb)
    area = (x2-x1) * (y2-y1)

    if current_target is None:
        with target_lock:
            target_id = tid
            current_target = tid
            print(f'Locked onto ID {target_id}')

    if tid == current_target:
        cx = (x1+x2) / 2.0
        best_angle = -1 * ((cx - frame_cx) /
frame_cx) * (HFOV / 2.0)
        best_area = area
        cv2.rectangle(frame, (x1,y1), (x2,y2),
(0,255,0), 2)
        cv2.putText(frame, f'TARGET #{tid}',
(x1, y1-10),
cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0,255,0), 2)
    else:
        cv2.rectangle(frame, (x1,y1), (x2,y2),
(128,128,128), 1)

```

```

        cv2.putText(frame, f'#{tid}',
                    (x1, y1-5),
cv2.FONT_HERSHEY_SIMPLEX,
                    0.5, (128,128,128), 1)

    elapsed = time.time() - start

    if best_angle is not None:
        cv2.putText(frame, f'Angle:{best_angle:.1f}',
                    (10,30), cv2.FONT_HERSHEY_SIMPLEX,
                    0.8, (0,255,0), 2)

        cv2.putText(frame, f'{1/elapsed:.0f} FPS
{elapsed*1000:.0f}ms',
                    (10,60), cv2.FONT_HERSHEY_SIMPLEX,
                    0.6, (0,200,255), 2)

    print(f'{elapsed*1000:.0f}ms | {1/elapsed:.0f}FPS
| '
        f'target:{current_target} |
angle:{best_angle}')

    # Send angle to Pi
    try:
        requests.post(ANGLE_URL,
                    json={'angle': best_angle},
                    timeout=0.05)
    except Exception:
        pass

    cv2.imshow('AVAA - Laptop YOLO', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

```

```
stream_alive = False  
cv2.destroyAllWindows()
```